Chapter 6

Search orders

Chapter 6. Search orders in CSPs

6.1 Introduction

6.2 Ordering of Variables in Searching

6.3 Ordering of Values in Searching

6.4 Ordering of Inferences in Searching

6.1 Introduction

Important issue - ordering in which the variables are labelled and the ordering in which the values are assigned to each variable.

Decisions in these orderings could affect the efficiency of the search strategies significantly.

The ordering in which the variables are labelled and the values chosen could affect the number of backtracks required in a search, which is one of the most important factors affecting the efficiency of an algorithm.

When the compatibility checks are computationally expensive, the efficiency of an algorithm could be significantly affected by the ordering of the compatibility checks.

6.2 Ordering of Variables in Searching

The following are some of the ways in which the ordering of the variables could affect the efficiency of a search:

(a) In lookahead algorithms, failures could be detected earlier under some orderings than others;

(b) In lookahead algorithms, larger portions of the search space can be pruned off under some orderings than others;

(c) In learning algorithms, smaller nogood sets could be discovered under certain orderings, which could lead to the pruning of larger parts of a search space;

(d) When one needs to backtrack, it is only useful to backtrack to the decisions which have caused the failure; Backtracking to the culprit decisions involves undoing some labels. Less of this is necessary in some orderings than others.

Heuristics for ordering the variables below. This is by no means an exhaustive list:

(1) the *minimal width ordering* (MWO) heuristic — by exploiting the topology of the nodes in the primal graph of the problem, the MWO heuristic orders the variables before the search starts. The intention is to reduce the need for backtracking;

(2) the *minimal bandwidth ordering* (MBO) heuristic — by exploiting the structure of the primal graph of the problem, the MBO heuristic aims at reducing the number of labels that need to be undone when backtracking is required;

(3) the *fail first principle* (FFP) — the variables may be ordered dynamically during the search, in the hope that failure could be detected as soon as possible;

(4) the *maximum cardinality ordering* (MCO) heuristic — MCO can be seen as a crude approximation of MWO.

6.2.1 The Minimal Width Ordering Heuristic

The minimal width ordering (MWO) of variables is applicable to problems in which some variables are constrained by more variables than others.

The heuristic is to first give the variables a total ordering which has the minimal width, and then label the variables according to that ordering.

Roughly speaking, the strategy is to leave those variables which are constrained by fewer other variables to be labelled last, in the hope that less backtracking is required.

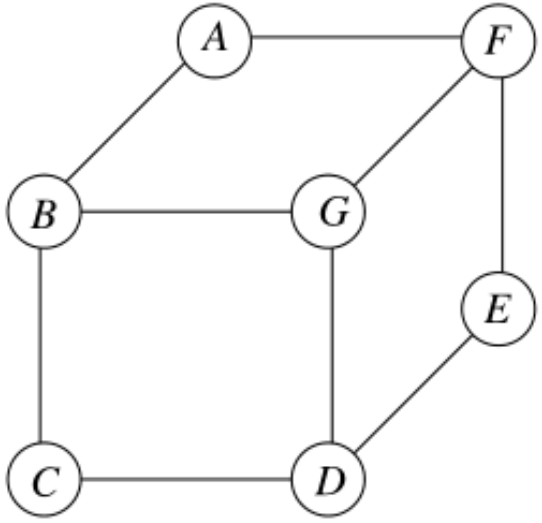
Definitions:

Given a total ordering < on the nodes of a graph, the width of a node v is the number of nodes before v (according to the ordering <) and adjacent to v.

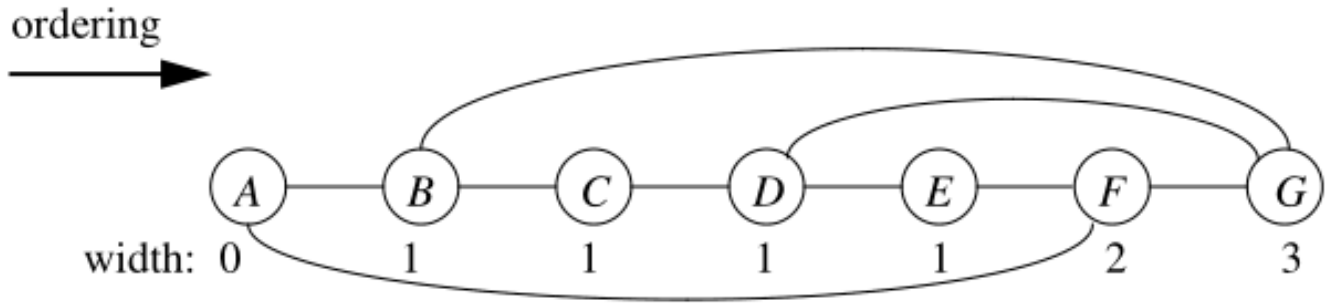
The width of an ordering is the maximum width of all the nodes under that ordering.

The width of the graph is the minimal width of all possible orderings.

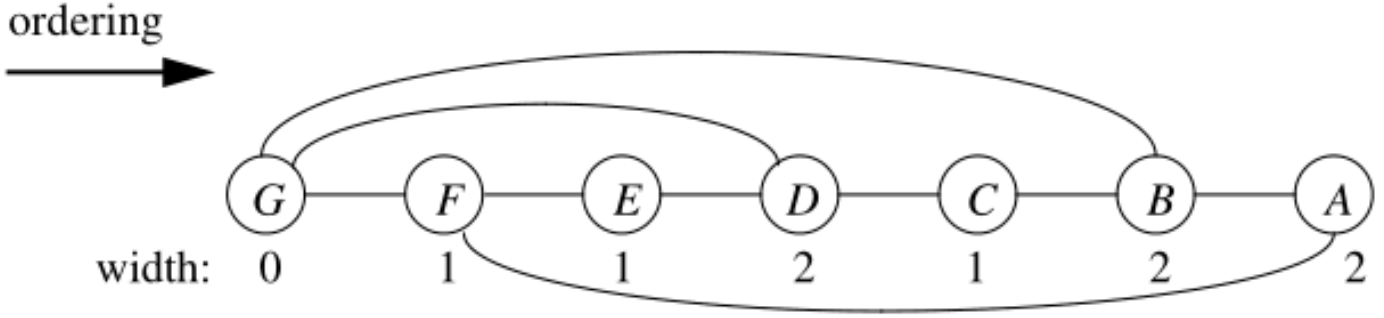
Figure 6.1 Example of a constraint graph with the width of different orderings shown



(a) A constraint graph to be labelled



(b) Width of the nodes given the order *A*, *B*, *C*, *D*, *E*, *F*, *G* (the width of each node is shown in italic)



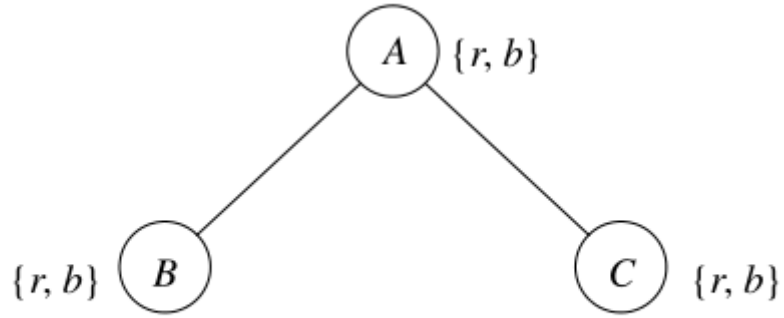
(c) Width of the nodes given the order *G*, *F*, *E*, *D*, *C*, *B*, *A* (the width of each node is shown in italic)

By labelling the variables under an ordering with a smaller width, the chance of backtracking can be reduced.

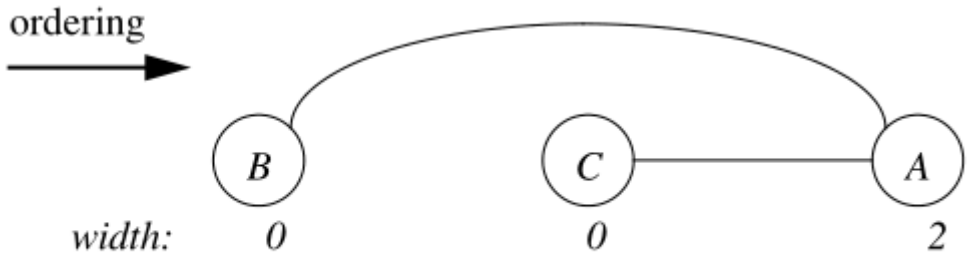
This is because the variables which have more unlabelled variables depending on them are labelled first.

So the variables at the front of the ordering are in general more constrained by other variables and the variables at the back normally have more freedom in the values that they can take.

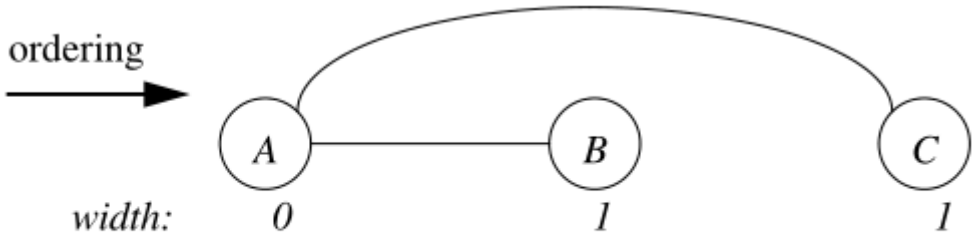
Figure 6.2 Example illustrating the significance of the search ordering: searching in the ordering shown in (b) may need backtracking (depending on the choice of values), but searching in the ordering shown in (c) and (d) never needs backtracking



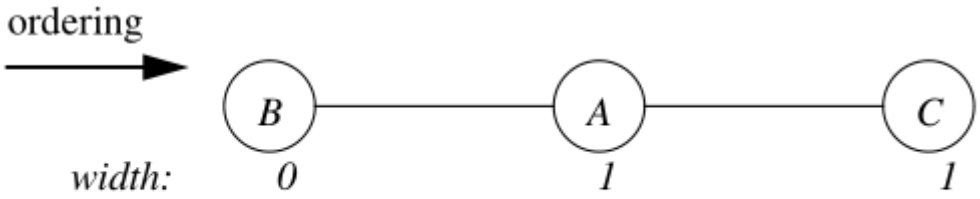
(a) The constraint graph to be searched



(b) Width of the ordering (B, C, A) is 2

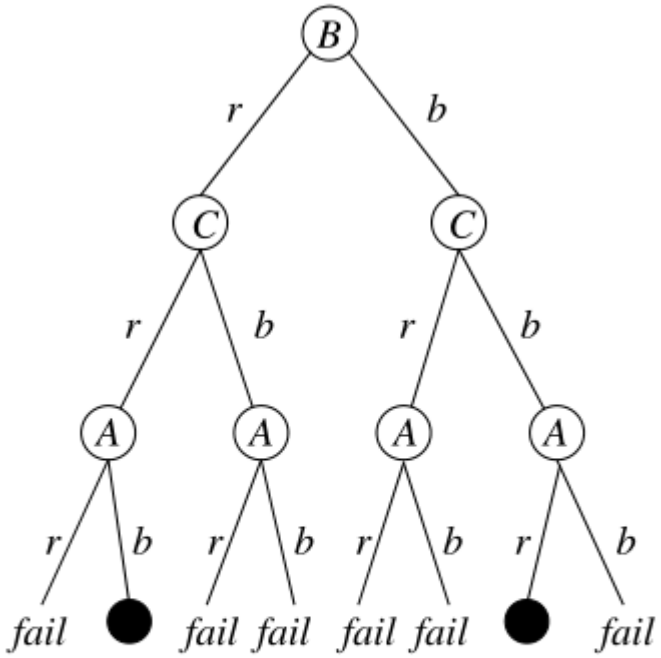


(c) Width of the search ordering (A, B, C) is 1

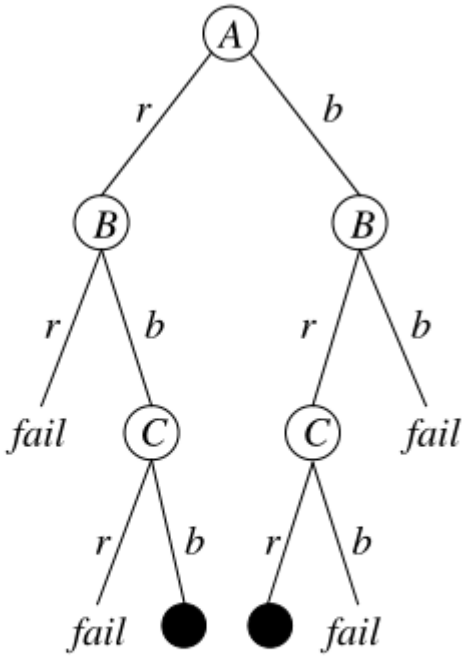


(d) Width of the search ordering (B, A, C) is 1

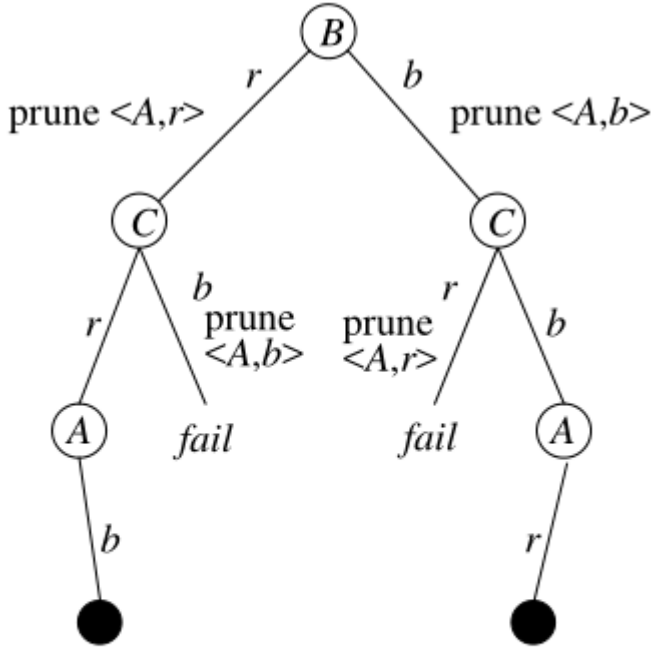
Figure 6.3 The search space explored by BT and FC in finding all solutions for the colouring problem in Figure 6.2(a) (● = solution)



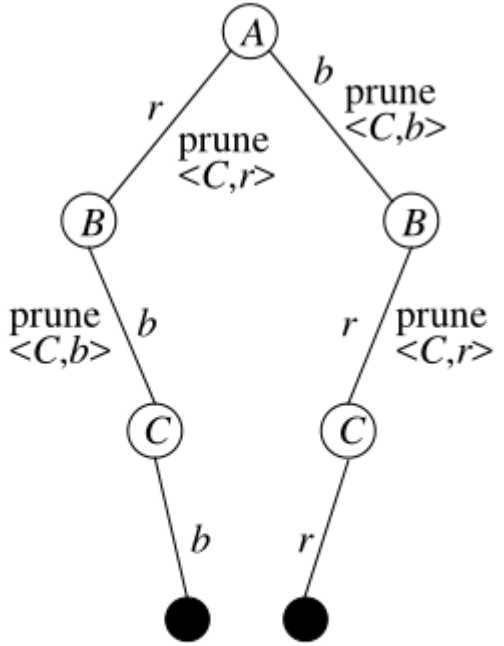
(a) The search space explored by BT under the ordering (*B, C, A*); number of branches explored: 14



(b) The search space explored by BT under the ordering (*A, B, C*); number of branches explored: 10



(c) The search space explored by FC under the ordering (*B*, *C*, *A*); number of branches explored: 8



(d) The search space explored by FC under the ordering (*A*, *B*, *C*); number of branches explored: 6

Theorem 6.1 (mainly due to Freuder, 1982)

Given a general CSP:

(i) A depth first search ordering is backtrack-free if the level of strong kconsistency in the problem (k) is greater than the width of the corresponding ordered constraint graph:

$\forall$ csp((*Z*, *D*, *C*)): ($\forall$ <: total_ordering(*Z*, <):
strong *k*-consistent((*Z*, *D*, *C*)) $\wedge$ width(G(*Z*, *D*, *C*), <) < *k* $\Rightarrow$
backtrack-free((*Z*, *D*, *C*), <)

(ii) There exists a backtrack-free depth first search ordering for the problem if the level of strong k-consistency in the problem (k) is greater than the width of the constraint graph.

$\forall$ csp((*Z*, *D*, *C*)):
strong *k*-consistent((*Z*, *D*, *C*)) $\wedge$ width(G(*Z*, *D*, *C*)) < *k* $\Rightarrow$
($\exists$ <: total_ordering(*Z*, <): backtrack-free((*Z*, *D*, *C*), <)

Theorem 6.1 extends the results of Theorem 3.1. It not only explains the motivation for achieving consistency, it also indicates the maximum k that strong k-consistency needs be achieved in order to make searches backtrack-free. It suggests that if a constraint graph has width w, then we should never need to achieve strong k-consistency for any $k > w + 1$. When $k \leq w + 1$, backtracking may be required. In general, the smaller $(w - k)$ is, the less backtracking can be expected.

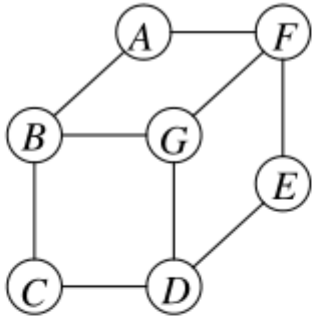
Theorem 6.2

A connected constraint graph (with more than one node) has width 1 if it is a tree.
$\forall$ graph(G): width(G) = 1 $\Leftrightarrow$ tree(G)

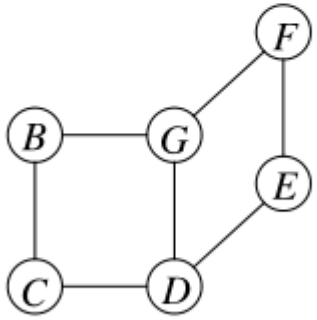
```
PROCEDURE Find_Minimal_Width_Ordering( (V, E) )
/* (V, E) is a graph where V, E are the set of nodes and edges,
    respectively */
BEGIN
    Q = ( );            /* Q is initialized to an empty sequence */
    REPEAT
        N ← the node in V joined by the least number of edges in E;
        /* in case of a tie, make an arbitrary choice */
        V ← V – {N};
        remove all the edges from E which join N to other nodes in V;
        Q ← N:Q;              /* make N the head of the sequence */
    UNTIL (V = { });
    return(Q);    /* Q = sequence of nodes in minimal width ordering */
END /* of Find_Minimal_Width_Ordering */
```

The Find_Minimal_Width_Ordering procedure returns a sequence which has the minimal width of the graph, i.e. the width of the returned ordering is the width of the constraint graph.

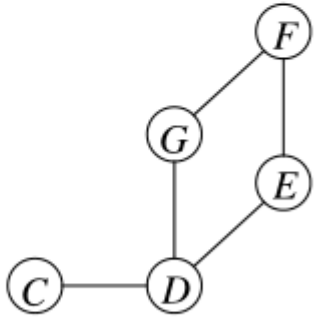
Figure 6.4 Example illustrating the steps taken by the Find_Minimal_Width_Ordering algorithm; the ordering found is: (*G, F, E, D, C, B, A*)



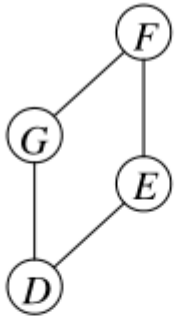
(a) The constraint graph to be labelled



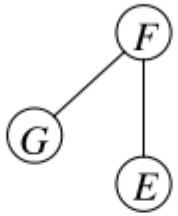
(b) Node *A* removed; ordering: (*A*)



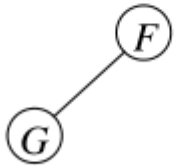
(c) Node *B* removed; ordering: (*B, A*)



(d) Node C removed;
ordering: (C, B, A)



(e) Node D removed;
ordering: (D, C, B, A)



(f) Node E removed;
ordering: (E, D, C, B, A)



(g) Node F removed;
ordering: (F, E, D, C, B, A)

Let the time to find the degree of a node be constant. Let n be the number of nodes in the graph. Therefore, the time complexity of the algorithm is $O(n^2)$.

max-degree ordering

- similar ordering method which simply orders the nodes by their degrees.
- motivation is the same as the MWO, i.e. to find an ordering which could reduce the need to backtrack.
- max-degree ordering is an approximation of the MWO, but requires less computation.

6.2.2 The Minimal Bandwidth Ordering Heuristic

The **minimal bandwidth ordering** (MBO) heuristic of variables is applicable to CSPs in which the constraint graph is not complete. Like the MWO heuristic, the MBO heuristic is used for preprocessing: the variables are given a total ordering with the minimal bandwidth (Definition 6-2 below) before search starts.

The intuition behind this heuristic is that the closer the constrained variables are placed to each other, the less distance one has to backtrack in case of failure.

Let h be an ordering of the nodes in a graph, and h maps every node v in the graph to the position that v is at under the ordering h.

For example, if the set of nodes is $\{a, b, c, d\}$, and the ordering h is (a, b, c, d), then $h(c) = 3$, because c comes third in the ordering.

Definition 6-1:
The **bandwidth of a node** v in an ordered graph is the maximum distance between v and any other node which is adjacent to v according to the ordering:

$\forall$ graph((V, E)):
bandwidth(v, (V, E), h) $\equiv$
MAX $|h(v) - h(w)| : w \in$ neighbourhood(v, (V, E))

Definition 6-2:
The **bandwidth of an ordering** h is the maximum bandwidth of all the nodes in the graph under the ordering h:

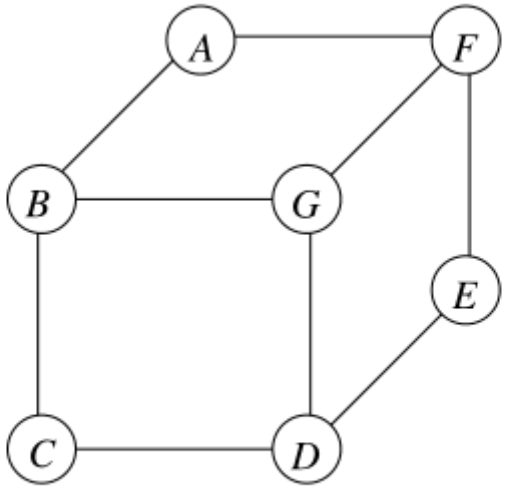
$\forall$ graph((V, E)):
(bandwidth((V, E), h) $\equiv$
MAX bandwidth(v, (V, E), h)): $v \in V$) ■

Definition 6-3:
The **bandwidth of a graph** is the minimal bandwidth of all orderings in the graph:

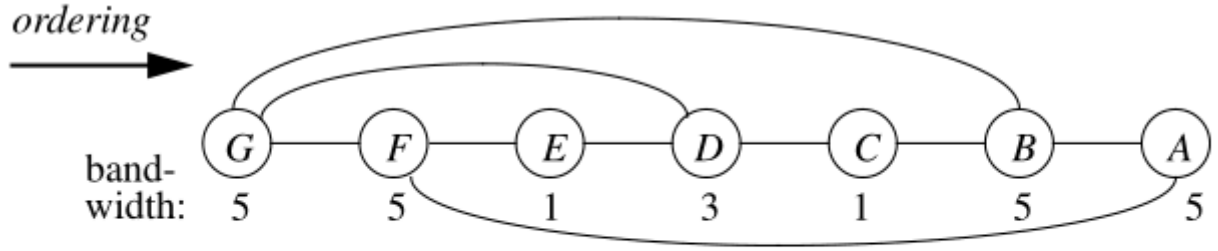
∀ graph((*V*, *E*)):
(bandwidth((*V*, *E*)) ≡
    MIN bandwidth((*V*, *E*), *h*): total_ordering(*V*, *h*)) ■

Figure 6.5(a) shows the same graph as Figure 6.1(a).
Figure 6.5(b) shows the bandwidth under the ordering shown in Figure 6.1(c).
Figure 6.5(c) shows an alternative ordering and the bandwidth.
In fact, careful analysis should reveal that this is the smallest bandwidth that one can get for the graph.

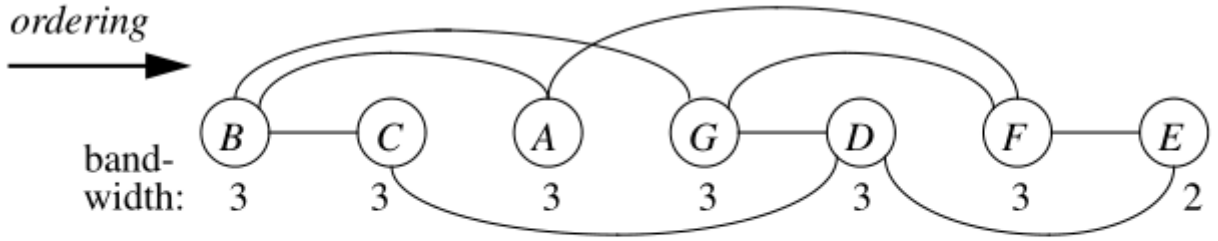
Figure 6.5 Example showing the bandwidth of two orderings of the nodes in a graph



(a) The graph to be labelled (same as that in Figure 6.1(a))



(b) The ordering (*G,F,E,D,C,B,A*) and its bandwidth (the bandwidth of each node is shown in italic, bandwidth of the ordering is 5)



(c) The ordering (*B,C,A,G,D,F,E*) and its bandwidth (the bandwidth of each node is shown in italic, bandwidth of this ordering is 3)

The concept of the MBO heuristic is used in ordering the variables before backtracking search starts. In general, the smaller the bandwidth of an ordering is, the sooner one could backtrack to relevant decisions in an algorithm which backtracks chronologically, such as the lookahead algorithms that we described in Chapter 5.

Theorem 6.3 (due to Zabih [1990])
For any graph G and any ordering < on its nodes, the bandwidth of G under < is always greater than or equal to the width of G under <.

∀ graph((*V*, *E*)):
(∀ <: total_ordering(*V*, <): bandwidth((*V*, *E*), <) ≥ width((*V*, *E*), <)

For any graph (*V*, *E*) and ordering <, let *b* = bandwidth((*V*, *E*), <). For any node *v*, all nodes before and adjacent to *v* must be within a distance of *b* according to the ordering <. Therefore, there can at most be b nodes before and adjacent to *v*.

Theorem 6.4 (due to Zabih [1990])
For any CSP P, if (V, E) is its primal graph and $<$ is a total ordering of the nodes V, then the bandwidth of (V, E) under $<$ is always greater than the induced-width of P under $<$.

$\forall$ graph((V, E)): ($\forall$ $<$: total_ordering(V, $<$):
bandwidth((V, E), $<$) $\geq$ induced-width((V, E), $<$)

Given any CSP P, if the nodes of its constraint graph (V, E) are given an ordering $<$, let b = bandwidth((V, E), $<$). For any node v in V, the Adaptive-consistency procedure will only add edges between the nodes before and adjacent to v. Since all the nodes before and adjacent to v are within a distance of b to v, the added edges between them will not increase the bandwidth of the induced graph. So the bandwidth of the induced graph is the same as bandwidth((V, E), $<$), which is greater than or equal to the width of the induced graph.

If the bandwidth of a graph is b, then a minimal bandwidth ordering can be found in both time and space $O(n^b)$, where n is the number of variables in the problem.

Empirical results supports the effectiveness of the MBO heuristic [Zabi90]. Tested on the graph-colouring problem alone, there is positive correlation between the bandwidth of the ordering and the size of the tree searched by a chronological backtracking strategy.

Saxe [1980] presented an algorithm with time and space complexity $O(n^{k+1})$ for determining whether a graph (with n nodes) has bandwidth k for any given integer k. This algorithm was improved by Gurari & Sudborough [1984], who presented an algorithm with time and space complexity $O(n^k)$. Their algorithm requires the graph to be connected.

Definition 6-4:
A **partial layout** of a graph G is a total ordering of a subset of the nodes in G:

$\forall$ graph((V, E)): ($\forall$ $Z \subseteq V$:
($\forall$ <: total_ordering(Z, <): partial_layout((Z, <), (V, E)))) ■

Definition 6-5:
Given a partial layout (Z, <) of a graph (V, E), a **dangling edge** is an edge which joins a node in Z to a node which is in V but not in Z:

$\forall$ graph((V, E)):
($\forall$ $Z \subseteq V$: ($\forall$ <: total_ordering(Z, <):
    ($\forall$ $(a, b) \in E$: dangling_edge((a, b), (Z, <), (V, E)) ≡
        ($a \in Z \wedge b \in V \wedge b \notin Z$)))) ■
The fact that Z is a subset of V and < is a total ordering of Z implies that (Z, <) is a partial layout of (V, E).

Definition 6-6:
A **conquered node** is a node in a partial layout which is not joined by any dangling edges:

$\forall$ graph((V, E)):
($\forall$ $Z \subseteq V$: ($\forall$ <: total_ordering(Z, <):
($\forall$ $a \in Z$: conquered_node(a, (Z, <), (V, E)) $\equiv$ $\forall$ $(a, b) \in E$: $b \in Z$)))■

Definition 6-7:
If (V, E) is a graph of which (Z, <) is a partial layout, then an **active node** in (Z, <) is a node which is adjacent to some nodes in V which are not in Z:

$\forall$ graph((V, E)):
($\forall$ $Z \subseteq V$: ($\forall$ <: total_ordering(Z, <):
($\forall$ $a \in Z$: active_node(a, (Z, <), (V, E)) $\equiv$ $\exists$ $(a, b) \in E$: $b \notin Z$))) ■

Observe that if a partial layout can be extended to an ordering of all the nodes in the graph with bandwidth ≤ k, the following must be true:

(1) The bandwidth of the partial layout is less than or equal to k;

(2) for all edges (x, y), if x is in the partial layout and y is not, then x must be among the last k elements in the partial layout (otherwise the distance between x and y in any ordering extended from this partial layout must be greater than k).

Therefore, we can focus on the last k elements of the partial layout. Furthermore, if $(v_1, v_2, ..., v_k)$ are the last k elements in the partial layout, and all nodes $v_1, v_2, ..., v_i$, where $i \leq k$, have no dangling edges, then we can separate all the nodes into three sets: the conquered nodes, the active nodes, and the unordered nodes.

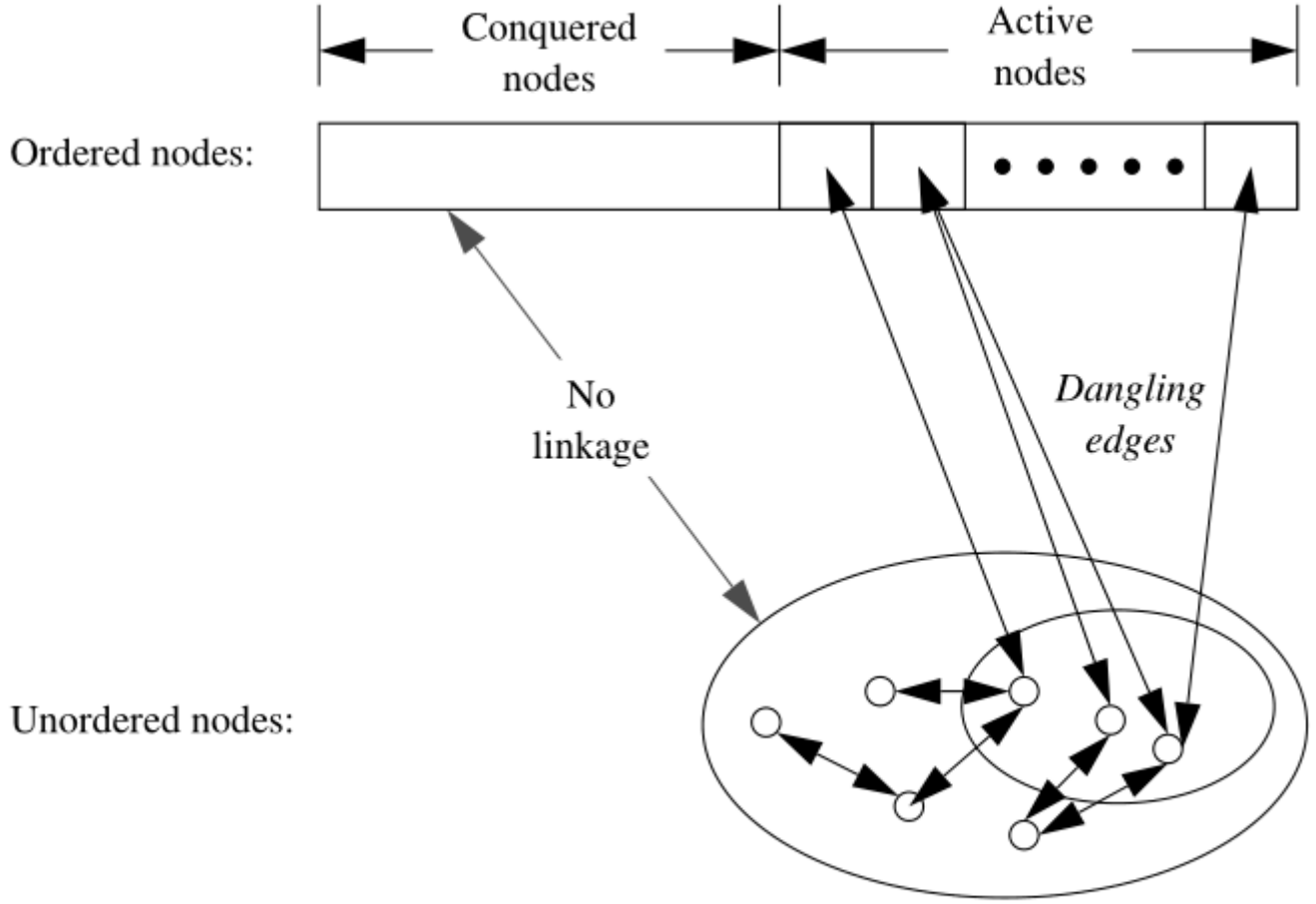


Figure 6.6 Node partitioning in bandwidth determination

The set of all active nodes plus the ordering under which they are defined is

called the active region. Since the graph is connected (by assumption), the active region plus the dangling edges together determines the conquered nodes and unordered nodes — all unordered nodes are either connected by some dangling edges, or adjacent to some other unordered nodes.

Based on these observations, the Determine_Bandwidth algorithm works by continuously extending the partial layout and updating the pair (r, d), where r is the active region and d is the set of dangling edges, until d is empty or it is provable that no ordering of bandwidth $\leq k$ can be generated. It makes use of two data structures:

- a first in first out (*fifo*) queue Q whose elements are (r, d) pairs, each of which representing a partial layout;
- a boolean array T, with one element per each (r, d) pair, recording whether this pair has been processed.

For any pair (r, d) and any node s, where $r = (v_1,v_2,...,v_i)$ is an active region and d

is a set of dangling edges for r, the procedure *Update_active_area((r, d), s)* below returns a new pair (r',d') where:

(a) r' is the sequence r with s appended to the end of it, and the sub-subsequence (v_1, v_2, ..., v_j) removed from it, where $j \leq i$ and all the nodes v_1, v_2, ..., v_j have no dangling edges in d except those which join them to s;

(b) d' is the set of dangling edges for the nodes in r'.

```
PROCEDURE Update_active_area( (r, d), s )
/* r = (v1, v2, ..., vi) */
/* Update_active_area appends a new node s to the end of r, and dis-
    cards all the nodes at the front of r which are no longer adjacent to
    any unordered nodes (via dangling edges) after s is added. */
BEGIN
    FOR each e in d DO
        IF (s is an end point of e) THEN d ← d – {e};
    j ← 1;
    WHILE ((vj is NOT joined by any edge in d) AND (j ≤ i)) DO
        j ← j + 1;
    FOR each edge e' in the graph which connects s DO
        IF (e' does not join s to any node in r) THEN d ← d + {e'};
        /* s will never be adjacent to any conquered nodes */
    return( (vj, vj+1, ..., vi, s), d );
END /* of Update_active_area */
```

```
PROCEDURE Plausible( (r,d), k )
/* r = (v1, v2, ..., vi), which is a (possibly empty) sequence of nodes */
/* d = nonempty set of dangling edges */
/* Plausible checks to see if r can possibly be extended to an ordering
    with bandwidth ≤ k: return False if any node in r has more dangling
    edges than limit, return True otherwise */
BEGIN
    IF ( |r| > k) THEN return(False)
    ELSE BEGIN
                Limit ← k – |r| + 1; j ← 1;
                    /* look at the first node in r; set its limit */
                WHILE (j < |r|) DO
                    BEGIN
                        IF (vj is joined by more than Limit edges in d)
                        THEN return(False)

                        ELSE BEGIN Limit ← Limit + 1; j ← j + 1; END
                    END
                return(True);
            END
END /* of Plausible */
```

```
PROCEDURE Determine_Bandwidth((V, E), k)
/* to determine whether there exists an ordering for the nodes of the
    graph (V, E) which bandwidth is ≤ k */
/* r and d are active regions and dangling edges, respectively, Q is a
    fifo queue of (r, d) pairs, T is a boolean array which records the
    (r, d) pairs processed */
BEGIN
    add ((), { }) to Q;
    set all elements in T to False;
    WHILE (Q ≠ { }) DO        /* basically a breadth-first search */
        BEGIN
            remove the head (r, d) from Q;
            /* r = (v1, v2, ..., vi), which is a (possibly empty) sequence of
                nodes */
            IF (r is a sequence of k nodes)
            THEN BEGIN
                        find any s such that (s, v1) ∈ d;
                        /* note that (s,v1) is the same object as (v1,s);
                            Update_active_area guarantees the exist-
                            ence of such s */
                        (r', d') ← Update_active_area( (r, d), s );
                        IF (d' = { }) THEN return(True);
                        ELSE IF (Plausible((r', d'), k) AND NOT T((r', d')))
                        THEN BEGIN
                                    T((r', d')) ← True; add (r', d') to end of Q;
                            END
                    END /* of then */
```

```
        ELSE FOR each unordered node s DO
                BEGIN
                    (r', d') ← Update_active_area( (r, d), s );
                    IF (d' = { }) THEN return(True);
                    ELSE IF (Plausible((r', d'), k) AND NOT T((r', d')))
                    THEN BEGIN
                                T((r', d')) ← True; add (r', d') to end of Q;

                            END
                END
    END /* of WHILE loop */
  return(False);
END /* of Determine_Bandwidth */
```

Update_active_area is the only procedure which adds and removes nodes from the active region. When a node is appended to the end of the input active region, all the edges which join it to other nodes, apart from those which join it to nodes that are already in the active region, will be added into the set of dangling edges. When a node is removed from the active region, it ensures that it is connected by no dangling edges. Therefore, Update_active_area guarantees that all conquered nodes are not adjacent to any unordered nodes. On the other hand, Update_active_area removes any node from the front of the active region which is no longer joined by any dangling edges after the new node is added into the region.

Determine_Bandwidth basically performs a breadth-first search in the space of active-region-dangling-edges pairs.

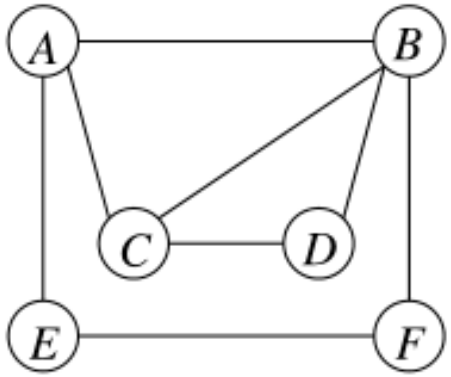
To find an ordering with the minimal bandwidth for a graph, one may call Determine_Bandwidth iteratively, increasing k by 1 at a time.

Gurari & Sudborough assume that the unordered nodes are computed rather than explicitly recorded in their analysis of space complexity.

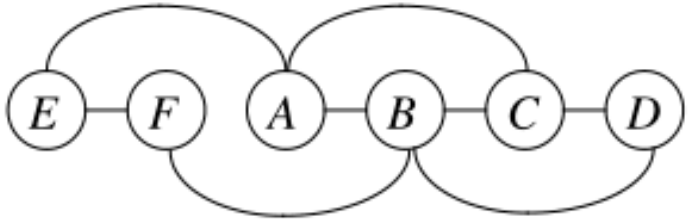
For a connected graph, all unordered nodes are accessible from some active nodes via the dangling edges. So y is an unordered node if and only if there exists a node x in the active area such that either (a) (x, y) is a dangling edge; or (b) there exists a path $(x, v_1, v_2, ..., v_i, y)$ in the graph, such that (x, v_1) is a dangling edge and all $v_1, v_2, ..., v_i$ are unordered nodes.

For a graph with n nodes, the time and space complexity of Determine_Bandwidth is $O(n^k)$

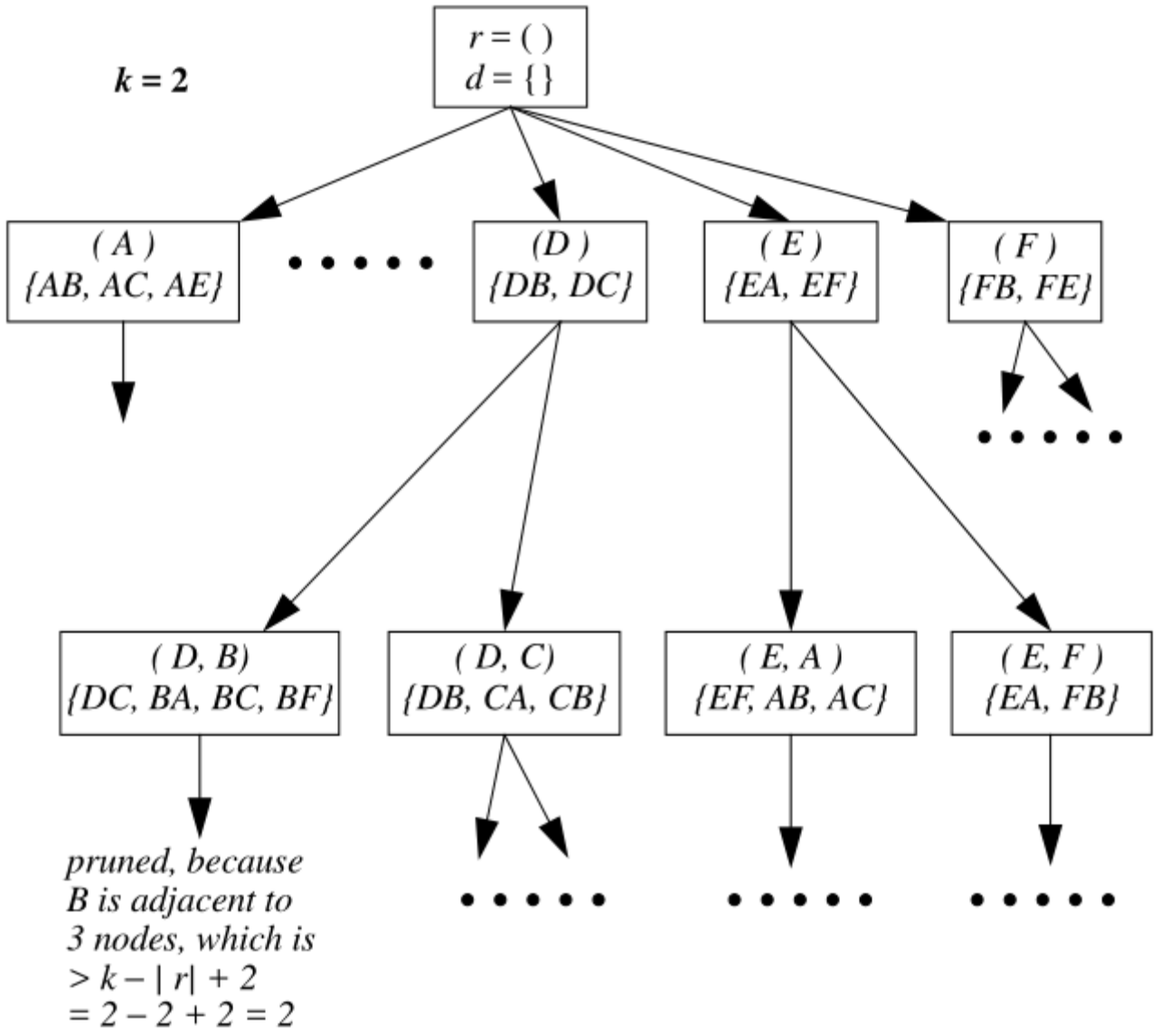
Figure 6.7 Example showing the space searched by Determine_-Bandwidth



(a) Graph to be processed



(b) Ordering with bandwidth = 2



(c) Part of the search tree explored by Determine_Bandwidth when $k = 2$

6.2.3 The Fail First Principle

The **Fail First Principle** (FFP) is a general heuristic for searching. It suggests that the task which is most likely to fail should be performed first. This heuristic aims at recognizing dead-ends as soon as possible so that search effort can be saved.

According to this strategy, the next variable to be labelled should be the variable which is the most constrained. The level difficulty in labelling a variable can be measured in different ways, one simple measure being the size of the domain. Under this measure, the variable which has the smallest domain should be labelled next.

The FFP is being employed by the constraint programming language CHIP, and impressive results have been reported.

In a simple backtracking algorithm such as the BT, the domain of the variables are static. Therefore, applying the FFP means sorting the variables in ascending order according to their domain size before search starts.

When the FFP is used together with lookahead algorithms, the ordering becomes dynamic. In a lookahead algorithm, after a variable is labelled constraints are propagated and values are possibly removed from the domains of unlabelled variables and the domain size of unlabelled variables could change dynamically. Therefore, when the FFP is applied, the search order must be determined dynamically. After assigning a value to each variable and propagating the constraints, the domains of all the unlabelled variables are compared and the variable which has the smallest domain will be selected.

Despite its simplicity, the FFP has been demonstrated to be quite effective in improving search efficiency. The FFP is effective because with it, one has a better chance of detecting failure sooner.

6.2.4 The maximum cardinality ordering

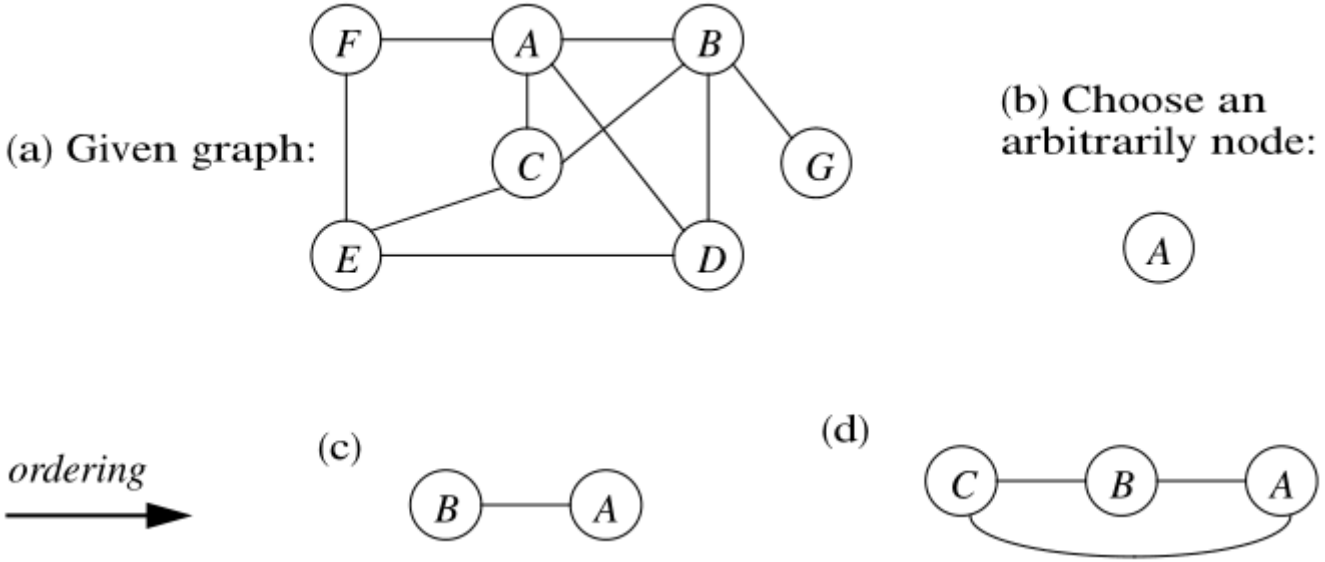
The **max-cardinality ordering** (MCO) heuristic can be seen as a crude approximation of the MWO heuristic. Although this ordering itself has been shown to be effective in certain problems, it is mentioned here mainly because it has useful properties for the tree-clustering method (Chapter 7).

The MCO can be obtained by picking the nodes in reverse order using the following step. To start, an arbitrary node is made the last node of the ordering. Then among all the unordered nodes, the one which is adjacent to the maximum number of already ordered nodes will be made the last, with ties broken

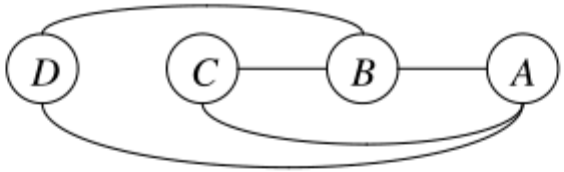
```
PROCEDURE Max_cardinality(V, E)
/* given a graph (V, E), return a maximum cardinality ordering of the
    nodes V. "Ordering" here is an array of nodes in V */
BEGIN
    N ← number of elements in V;
    Ordering[N] ← an arbitrary element of V; V ← V – Ordering[N];
    FOR i = N – 1 to 1 by –1 DO
        BEGIN
            Ordering[i] ← node in V which is adjacent to the maximum
                number of nodes between Ordering[i + 1] and Order-
                ing[N];
            V ← V – Ordering[i];
        END
    return(Ordering);
END /* of Max_cardinality */
```

If we assume that finding the node which is adjacent to the maximum number of ordered nodes takes a constant time, then the procedure Max_cardinality takes $O(n)$ time to compute, where n equals the number of nodes in the graph.

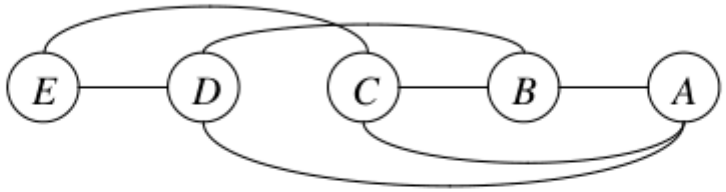
Figure 6.8 Example showing the steps taken by Max_cardinality



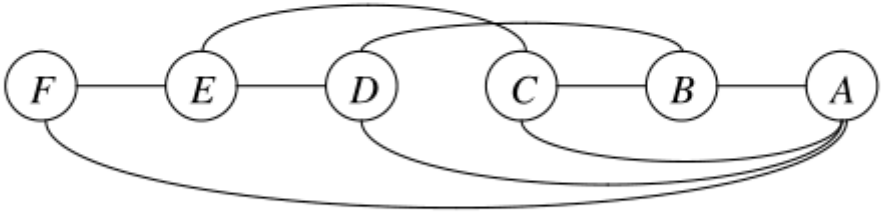
(e)



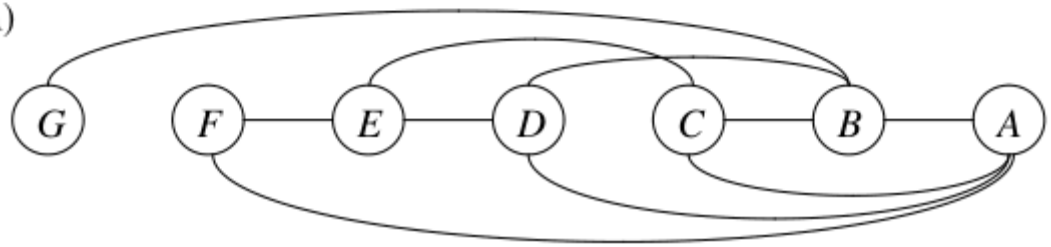
(f)



(g)



(h)



6.2.5 Finding the next variable to label

MWO, MBO, FFP and MCO are all heuristics for ordering the variables. In this section, we shall analyse the applicability of these heuristics under different circumstances, and study whether they can be applied together.

All MWO, MBO and MCO give the variables a fixed ordering before the search starts. One problem of doing so is that it does not take the domains of the variables into consideration. After constraint propagation, some variables could be more constrained than others, and therefore, one may benefit from labelling them first. The FFP, on the other hand, considers this, but does not consider the topology of the constraint graph.

In principle, the MWO heuristic aims at minimizing the need for backtracking, the MBO heuristic aims at minimizing the distance of chronological backtracking, and FFP aims at recognizing failures sooner. The relative efficiency of MWO, MBO, MCO and FFP are problem dependent. Here we shall give a crude guideline of the situations under which they may be effective.

In principle, the MWO heuristic may be useful for CSPs where:

(1) the degree of the nodes in the constraint graph varies significantly; it will not help, for example, in the *N*-queens problem, as each node in the constraint graph has the same degree;

(2) a certain level of consistency is maintained in the graph; in this case, it is worth finding out whether a backtrack-free search ordering can be established (Theorem 6.1).

The MBO heuristic may be useful for CSPs where no node in the constraint graph has high degrees. This is because the maximum degree among the nodes dictates the lower-bound of the minimal bandwidth of the graph. In general, the fewer edges a graph has, the smaller its minimal bandwidth is likely to be.

The FFP may be useful in problems where:

(1) the domain size of the variables varies significantly;

(2) constraints are tight, hence the domain sizes of the unlabelled variables could change significantly in lookahead algorithms;

Point (2) suggests that the FFP is especially effective when used with lookahead algorithms.

The MCO can be seen as a crude approximation of the MWO, which requires $O(n)$ time to compute, where n is the number of variables in the problem.

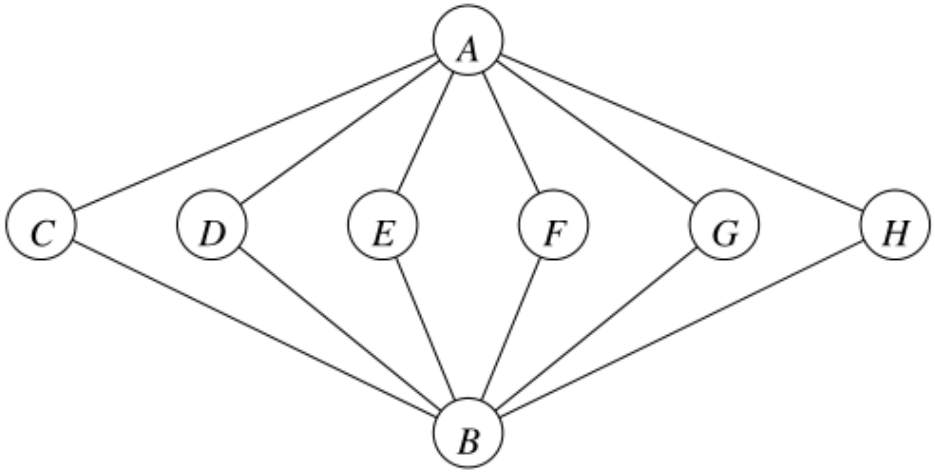
One could attempt to combine the MWO and MBO heuristics. It is not always possible to find orderings which minimize both the width and bandwidth simultaneously.

It is possible to combine the MWO and MBO heuristics with FFP. One way in which to do so is to employ FFP to order the variables dynamically, and in case of ties (i.e. several variables having the same domain size), use the principles in the MWO or MBO heuristics to select the next variable to label.

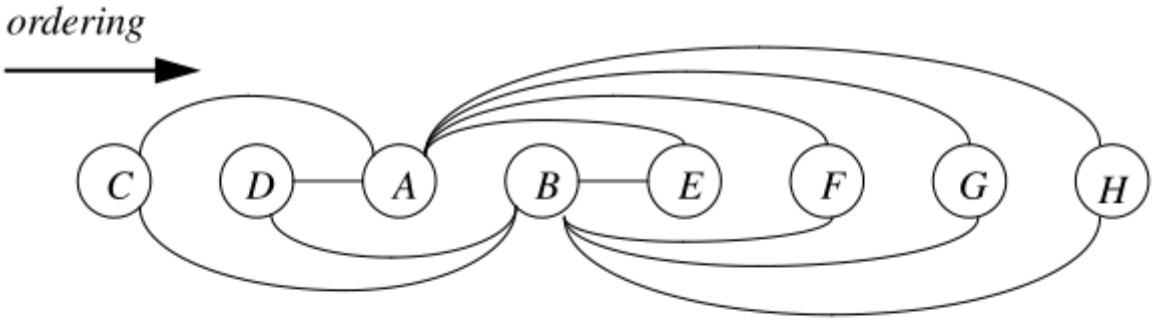
Another possible approach is to employ the MWO or MBO heuristic to order the variables before labelling, and in case of ties in running the procedures Find_Minimal_Width_Ordering or Find_Minimal_Bandwidth_Ordering, pick the variable which has a smaller domain size. Obviously, the justification of the overhead involved is problem- or domain-dependent.

It may worth emphasizing that MWO, MB, FFP and MCO are general heuristics only. Given a particular application, domain knowledge should always be looked at, because sometimes effective domain-specific heuristics may be available. Besides, the heuristics described in this chapter have not considered the tightness of individual constraints.

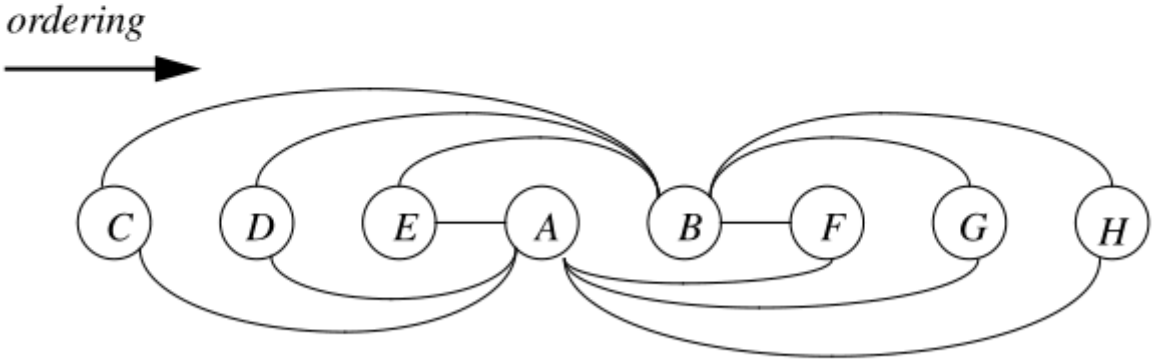
Figure 6.9 Example of a constraint graph in which the minimal width and minimal bandwidth cannot be obtained in the same ordering (the width of the graph is 2, and the bandwidth of the graph is 4)



(a) The constraint graph to be ordered



(b) An ordering with minimal width, but not minimal bandwidth (wdith = 2, bandwidth = 5)



(c) An ordering with minimal bandwidth, but not minimal width (width = 3, bandwidth = 4)

6.3 Ordering of Values in Searching

It has long been suggested that the efficiency of a search for general search problems can be greatly affected by the ordering in which we explore the branches (e.g. see Nilsson, 1980). Therefore, the efficiency of a search algorithm for solving CSPs can be affected by the ordering under which the values are selected for each variable.

When picking the next variable to label, we pick the most constrained one first because if it can be established that this variable cannot be consistently labelled, there is no need to attempt to label the other variables. That is the rationale behind not only the FFP heuristic, but also the MWO heuristic. On the other hand, when picking the next value to assign to a variable, we want to pick the value which is most likely to succeed, because failure in this case would cause backtracking.

Heuristics for ordering the values may help one to find the first solution more efficiently if the branches which have a better chance of reaching a solution can be identified and searched first. However, unless learning algorithms are employed, ordering of the values does not help one to prune off any search space. So unless learning takes place, ordering of the values is only useful for finding single solutions.

In ordering the values, one would like to put those values which are most promising at the front. However, there may be many ways to evaluate the likelihood of success in a value. One heuristic is called the min-conflict heuristic, which basically orders the values according to the conflicts which they are involved with the unlabelled variables. Basically, this heuristic can be used together with all those algorithms described in the Chapter 5.

The *Informed-backtrack* algorithm below makes use of the min-conflict heuristic. For simplicity, only binary constraints are considered in this algorithm. It starts with two sets: LABELS_LEFT and LABELS_DONE. LABELS_LEFT is initialized to a set of random assignments for all the variables, and LABELS_DONE is initialized to an empty set. Then the program starts to resolve any conflict that exists.

```
PROCEDURE Informed-Backtrack( Z, D, C );
BEGIN
    LABELS_LEFT ← { };
    FOR each variable x ∈ Z DO
        BEGIN
            pick a random value from D_x;
            add <x,v> to LABELS_LEFT
        END;
    InfBack( LABELS_LEFT, { }, D, C );
END /* of Informed-Backtrack */
```

```
PROCEDURE InfBack( LEFT, DONE, D, C );
/* Basically InfBack performs a depth first search. In each step, it
    attempts to replace an illegal label in LEFT */
BEGIN
    IF (there exists any set of incompatible labels in LEFT)
    THEN BEGIN
        x ← any variable which label <x,v> is in LEFT and is <x,v> is in
            conflict with some other labels;
        Queue ← Order_values( x, D_x, Labels_left, Labels_done, C );
        WHILE (Queue ≠ { }) DO
            BEGIN
                w ← first element in Queue; Delete w from Queue;
                DONE ← DONE + {<x,w>};
                Result ← InfBack( LEFT – {<x,v>}, DONE, D, C );
                IF (Result ≠ NIL) THEN return(Result);
            END;
        return(NIL);        /* all values in Queue tried but failed */
    END /* of THEN */
    ELSE return( LEFT  +  DONE );
END /* of InfBack */
```

```
PROCEDURE Order_values( x, D_x, LEFT, DONE, C )
/* Order_values sorts the values of x in ascending order of the
    number of labels in LEFT that these values have conflict with. A
    value is discarded if it is incompatible with any label in DONE */
BEGIN
    List ← { };
    FOR each v ∈ D_x DO
        BEGIN
            IF (<x,v> is compatible with all the labels in DONE)
            THEN BEGIN
                    Count[v] ← 0;   /* Let Count be an array of integers */
                    FOR each <y,w> in LEFT DO
                        IF NOT satisfies( (<x,v><y,w>), C_x,y )
                        THEN Count[v] ← Count[v] + 1;
                END
            List ← List + {v};
        END
    Queue ← the values in List ordered in ascending order of
        Count[v];
    return( Queue );
END /* of Order_values */
```

6.4 Ordering of Inferences in Searching

Compatibility checks are computationally expensive in certain applications. In such applications, the efficiency of the algorithm could be significantly affected by the number of compatibility checks being made. In lookahead algorithms, propagating the constraints imposed by the assignment of a value to a variable to the unlabelled variables involves making inferences. The higher the level of consistency one maintains, the more backtracking one could potentially avoid, but the more inferences one has to make. In algorithms which maintain a high level of consistency, the number of inferences that needs to be made significantly affect the search efficiency, but could be affected by the ordering in which they are made. The best known heuristic in this area is the Fail First Principle (FFP). For inference ordering, this principle suggests performing those inferences which are most likely to detect failure first. The sooner a dead-end is detected, the fewer inferences will need to be performed. Domain knowledge is normally required to evaluate the chance of an inference failing.